

Adaptive mesh refinement and parallelisation Euler AMReX

BCN: 8774Z

Cavendish Laboratory, Department of Physics, J J Thomson Avenue, Cambridge. CB3 0HE

Abstract

This paper implements a compressible Euler equation solver within the AMReX framework, achieving second-order accuracy using the MUSCL-Hancock finite volume scheme and some approximate riemann solvers. The impact of adaptive mesh refinement and MPI parallelization on computational efficiency is investigated. The solver is validated on Toro's standard Riemann problem in one and two dimensions, exhibiting second-order convergence in smooth regions and accurately capturing shock waves, rarefaction waves, and contact discontinuities. Applying AMR, a gradient and threshold-based labeling criterion concentrates the resolution at discontinuities, reproducing solutions with uniform fine meshes with significantly lower computational cost. MPI parallelization efficiency is tested using the Lax-Liu two-dimensional Riemann problem. Combining AMR with multi-core parallelization achieves a significant speedup compared to equivalent uniform meshes, and the impact of AMR parameters such as the refinement threshold on efficiency is examined.

1. Introduction

The compressible Euler equations describe the behavior of inviscid compressible flows and appear in a broad range of problems in computational fluid dynamics [1]. Finding accurate numerical solutions to the compressible Euler equations is computationally challenging due to the existence of discontinuous features such as shock waves and contact discontinuities.

In the present report, the numerical solution to the two-dimensional compressible Euler equations is presented. The numerical solution is implemented using the AMReX [2] computational software framework. The numerical method is a second-order accurate MUSCL-Hancock method with the HLLC Riemann solver [1] to advance the numerical solution in time. Adaptive mesh refinement [3] is also applied to dynamically concentrate computational effort around regions of high gradient magnitude such as shock waves and contact discontinuities, leaving smooth regions at coarse resolutions. The numerical accuracy of the numerical method is validated using the standard set of one-dimensional Riemann problem test cases [1] extended to two dimensions. The numerical method is then applied to the two-dimensional Riemann problem test case. The computational efficiency of the numerical method is analyzed through a timing study.

2. Methodology

In the next section, the entire framework of solving the second-order Euler equations with the help of the AMR technique will be discussed, including the fundamental equations involved, the numerical techniques employed, and the way the Euler equations can be solved with the help of the AMR technique.

2.1. Governing Equations:

One-dimensional compressible Euler equations

The one-dimensional Euler equations describe the motion of an inviscid compressible gas, expressing conservation of mass, momentum, and energy[1].

$$\partial_t \mathbf{U} + \partial_x \mathbf{F}(\mathbf{U}) = 0 \quad (1)$$

The conserved variable vector $\mathbf{U}$ and the flux vector $\mathbf{F}$ are respectively:

$$\mathbf{U} = \begin{pmatrix} \rho \\ \rho u \\ E \end{pmatrix} \quad \mathbf{F} = \begin{pmatrix} \rho u \\ \rho u^2 + p \\ u(E + p) \end{pmatrix} \quad (2)$$

To close the system of equations, we refer to the ideal gas law[1], where $\gamma = 1.4$ is the ratio of specific heats.

$$p = (\gamma - 1) \left(E - \frac{1}{2} \rho u^2 \right) \quad (3)$$

The eigenvalues of the flux Jacobian $\partial F / \partial U$ are $u - a$, u , and $u + a$, where $a = \sqrt{\gamma p / \rho}$ [1] denotes the local speed of sound. These eigenvalues determine the maximum wave speed and are used in the evaluation of the CFL condition.

Two-dimensional compressible Euler equations

The extension to two dimensions introduces an additional momentum equation and flux component in the y -direction[1]:

$$\partial_t \mathbf{U} + \partial_x \mathbf{F}(\mathbf{U}) + \partial_y \mathbf{G}(\mathbf{U}) = 0 \quad (4)$$

The conserved variable vector $\mathbf{U}$ and the flux vector $\mathbf{F}$ and $\mathbf{G}$ are respectively:

$$\mathbf{U} = \begin{pmatrix} \rho \\ \rho v_x \\ \rho v_y \\ E \end{pmatrix}, \quad \mathbf{F} = \begin{pmatrix} \rho v_x \\ \rho v_x^2 + p \\ \rho v_x v_y \\ v_x(E + p) \end{pmatrix}, \quad \mathbf{G} = \begin{pmatrix} \rho v_y \\ \rho v_x v_y \\ \rho v_y^2 + p \\ v_y(E + p) \end{pmatrix} \quad (5)$$

To close the system, the ideal gas law generalises to:

$$p = (\gamma - 1) \left(E - \frac{1}{2} \rho (v_x^2 + v_y^2) \right) \quad (6)$$

CFL Condition

To ensure numerical stability, the time step Δt is determined by the CFL condition [1]. The maximum Courant number C_{cfl} is defined as:

$$C_{\text{cfl}} = \frac{\Delta t S_{\text{max}}^n}{\Delta x} \quad (7)$$

Where S_{max}^n is the maximum wave speed across the domain at time level n , and $0 < C_{\text{cfl}} \leq 1$. The time step is then:

$$\Delta t = C_{\text{cfl}} \frac{\Delta x}{S_{\text{max}}^n} \quad (8)$$

In two dimensions, this is applied in both coordinate directions, taking the minimum:

$$\Delta t = C_{\text{cfl}} \cdot \min \left(\frac{\Delta x}{S_{\text{max}}^x}, \frac{\Delta y}{S_{\text{max}}^y} \right) \quad (9)$$

Where $S_{\text{max}}^x = |v_x| + a$ and $S_{\text{max}}^y = |v_y| + a$, with:

$$a = \sqrt{\frac{\gamma p}{\rho}} \quad (10)$$

Which is the local speed of sound.

2.2. Numerical Methods:

In this part, we first introduce Finite Volume Framework. Then, we will briefly describe some numerical methods that we used in this assignment.

Finite-Volume Framework

In the assignment, we take the baseline method to be the finite volume method, where the conservation law is integrated over each control volume. This ensures the conservation property by construction[1].

Take the x direction for example, the y direction is similar to the x direction. The interval $[x_0, x_1]$ is uniformly partitioned into N cells. The i th cell is defined as:

$$I_i = [x_{i-\frac{1}{2}}, x_{i+\frac{1}{2}}] \quad (11)$$

Where the cell width is:

$$\Delta x = x_{i+\frac{1}{2}} - x_{i-\frac{1}{2}} \quad (12)$$

And the cell center is:

$$x_i = \frac{1}{2}(x_{i-\frac{1}{2}} + x_{i+\frac{1}{2}}) \quad (13)$$

In the finite volume method, the unknowns are element averages, not point values:

$$\mathbf{U}_i^n = \frac{1}{\Delta x} \int_{I_i} \mathbf{U}(x, t^n) dx \quad (14)$$

In two dimensions, the domain is partitioned into a uniform Cartesian grid of cells with cell averages defined analogously.

$$I_{i,j} = [x_{i-\frac{1}{2}}, x_{i+\frac{1}{2}}] \times [y_{j-\frac{1}{2}}, y_{j+\frac{1}{2}}] \quad (15)$$

$\bar{F}_{i+\frac{1}{2}}$ is the time-averaged numerical flux at the cell interface $x_{i+\frac{1}{2}}$.

$$\bar{F}_{i+\frac{1}{2}} = \frac{1}{\Delta t} \int_{t^n}^{t^{n+1}} F(\mathbf{U}(x_{i+\frac{1}{2}}, t)) dt \quad (16)$$

By integrating Equation (1) over the control volume $I_i \times [t^n, t^{n+1}]$ and applying the divergence theorem, the update formula is obtained directly:

$$\mathbf{U}_i^{n+1} = \mathbf{U}_i^n - \frac{\Delta t}{\Delta x} (\bar{F}_{i+\frac{1}{2}} - \bar{F}_{i-\frac{1}{2}}) \quad (17)$$

To ensure numerical stability, the time step Δt is restricted by the CFL condition[1], in which $s_{\max}$ is the maximum wave speed and $C_{\text{cfl}} < 1$.

The Riemann Problem:

The Riemann problem is a form of initial value problem, consisting of a conservation law and a piecewise-constant initial condition with a single discontinuity located at $x = x_0$ [1]. It can be written as:

$$\frac{\partial \mathbf{U}}{\partial t} + \frac{\partial \mathbf{F}(\mathbf{U})}{\partial x} = 0 \quad \mathbf{U}(x, 0) = \begin{cases} \mathbf{U}_L, & x < x_0 \\ \mathbf{U}_R, & x > x_0 \end{cases} \quad (18)$$

The Riemann Solver:

The solution of the Riemann problem can be carried out approximately or exactly. The approximate solvers include the HLL, HLLC, and Lax-Friedrichs solvers, which are computationally efficient but add artificial dissipation, see [1] for the details of the artificial dissipation. In the exact solver, the exact solution of all wave structures is obtained through iteration of the star-pressure p^* , as described in [1].

$$f(p^*, U_L, U_R) = f_L(p^*, U_L) + f_R(p^*, U_R) + (u_R - u_L) = 0 \quad (19)$$

where for each side $K \in \{L, R\}$:

$$f_K(p^*, u_K) = \begin{cases} (p^* - p_K) \sqrt{\frac{A_K}{p^* + B_K}}, & p^* > p_K \text{ (shock)} \\ \frac{2c_{s,K}}{\gamma - 1} \left[\left(\frac{p^*}{p_K} \right)^{\frac{\gamma-1}{2\gamma}} - 1 \right], & p^* \leq p_K \text{ (rarefaction)} \end{cases} \quad (20)$$

with $A_K = \frac{2}{(\gamma + 1)p_K}$ and $B_K = \frac{\gamma - 1}{\gamma + 1} p_K$. The root is found via Newton-Raphson iteration [7]:

$$p_{(m)}^* = p_{(m-1)}^* - \frac{f(p_{(m-1)}^*, U_L, U_R)}{f'(p_{(m-1)}^*, U_L, U_R)} \quad (21)$$

In this work, the exact solver is used as the primary choice, with HLL, HLLC and Lax-Friedrichs employed as a fallback near strong discontinuities.

Total Variation Diminishing (TVD) Schemes

A numerical scheme of the form:

$$u_i^{n+1} = H(u_{i-k_L}^n, \dots, u_{i+k_R}^n) \quad (22)$$

Is said to be Total Variation Diminishing (TVD) if[1]:

$$TV(u^{n+1}) \leq TV(u^n), \quad \forall n, \quad (23)$$

Where:

$$TV(u^n) = \sum_i |u_{i+1}^n - u_i^n|. \quad (24)$$

This ensures that there is no introduction of new local extrema during the time evolution. However, Godunov's theorem proves that any linear monotone scheme is at most first-order accurate[1]. In the TVD method, nonlinear limiters are used to attain second-order accuracy. This is the upper limit of the TVD method.

MUSCL-Hancock method

MUSCL Hancock is a second-order TVD finite volume method developed by Hancock. This method is a variation of Godunov's method with the use of piecewise linear reconstruction instead of piecewise constant reconstruction to obtain second-order accuracy in space as well as time [1].

To avoid spurious oscillations near discontinuities while preserving the second-order accuracy of the method in smooth regions, the slope limiter is applied to the piecewise linear reconstruction. Several slope limiters can be applied to the piecewise linear reconstruction to avoid spurious oscillations near discontinuities while preserving the second-order accuracy of the method in smooth regions. Some of the slope limiters applied to the piecewise linear reconstruction are the Minmod and Van Leer slope limiters[4], which can be seen in [1]; the MC slope limiter is a symmetric slope limiter given in terms of the ratio of successive gradients:

$$r_i = \frac{U_i - U_{i-1}}{U_{i+1} - U_i} \quad (25)$$

The limiter function is defined as[5]:

$$\phi^{MC}(r) = \max(0, \min(2r, 0.5(1+r), 2)) \quad (26)$$

The reconstructed slope is then:

$$\Delta_i = \phi^{MC}(r_i)(U_{i+1} - U_i) \quad (27)$$

In smooth regions where $r \approx 1$, the limiter returns a value close to 1, preserving second-order accuracy.

Near discontinuities where $r \leq 0$, the limiter returns zero, reducing the scheme locally to first order and suppressing spurious oscillations.

1D scheme:

The MUSCL Hancock scheme constructs the update in three steps[1]:

• Step I: Reconstruction

Within each cell I_i , a piecewise-linear reconstruction is performed using the limited slope Δ_i . The boundary-extrapolated values are:

$$U_i^L = U_i^n - \frac{1}{2}\Delta_i, \quad U_i^R = U_i^n + \frac{1}{2}\Delta_i \quad (28)$$

• Step II: Evolution

The boundary-extrapolated values are evolved by a half time step $\frac{1}{2}\Delta t$ using the governing flux:

$$\bar{U}_i^L = U_i^L + \frac{1}{2} \frac{\Delta t}{\Delta x} [F(U_i^L) - F(U_i^R)] \quad (29)$$

$$\bar{U}_i^R = U_i^R + \frac{1}{2} \frac{\Delta t}{\Delta x} [F(U_i^L) - F(U_i^R)] \quad (30)$$

Step III: Flux computation

The evolved states $\bar{U}_i^R$ and $\bar{U}_{i+1}^L$ form a piecewise-constant Riemann problem at each interface $x_{i+\frac{1}{2}}$, which is solved using Riemann solver to obtain the numerical flux $\mathbf{F}_{i+\frac{1}{2}}$. The full conservative update is then:

$$\mathbf{U}_i^{n+1} = \mathbf{U}_i^n - \frac{\Delta t}{\Delta x} (\mathbf{F}_{i+\frac{1}{2}} - \mathbf{F}_{i-\frac{1}{2}}) \quad (31)$$

2D Extension:

In 2D space, the extension of this scheme is achieved using an unsplit method, which calculates the fluxes in both the x and y directions simultaneously within a single time step, which is important for problems with oblique discontinuities. For transverse wave propagation, a Corner Transport Upwind (CTU) correction is used, which is an extension of the multidimensional upwind algorithm by Colella,[6] using a provisional half-step flux in one direction to correct the interface state before obtaining the flux in the other direction. The 2D conservative update is:

$$\mathbf{U}_{i,j}^{n+1} = \mathbf{U}_{i,j}^n - \frac{\Delta t}{\Delta x} (\mathbf{F}_{i+\frac{1}{2},j} - \mathbf{F}_{i-\frac{1}{2},j}) - \frac{\Delta t}{\Delta y} (\mathbf{G}_{i,j+\frac{1}{2}} - \mathbf{G}_{i,j-\frac{1}{2}}) \quad (32)$$

2.3. Adaptive Mesh Refinement

For compressible Euler equations, physical phenomena such as shock waves or contact discontinuities usually occupy only a small fraction of the computational domain. Therefore, if the entire mesh is refined uniformly, the computational effort spent on the smooth regions is essentially wasted because the solution is smooth almost everywhere, and the necessary resolution is already easily obtainable with a coarse mesh. Furthermore, the additional resolution does not supply any additional physical information, but the computational cost increases exponentially with the refinement level.

Adaptive mesh refinement (AMR)[3] is a technique to overcome the above problem by allowing the mesh to be locally refined to regions where high resolution is necessary. When applied correctly, the speedup of the computation can be substantial compared to the uniformly refined mesh. However, note that the accuracy of the results is always lower than the uniform mesh with the same refinement level, and unless the parameters are set correctly, the results may contain spurious oscillations or non-physical states near the mesh interfaces.

The computational domain is divided into a hierarchy of levels $\ell = 0, 1, \dots, L_{\max}$, where level 0 is the coarsest base grid with a cell size of Δx_0 , and each subsequent level is refined from the previous level by a refinement ratio of r in each dimension. Thus, the cell size at level ℓ is $\Delta x_\ell = \Delta x_0 / r^\ell$. At each level, the domain is covered by a collection of disjoint rectangles fully contained within the corresponding level. At any point in the domain, only the finest level contributes to the physical solution, with the information from the coarser levels in the overlapping region being used only as boundary conditions for the fine level[7].

Tagging Criterion and Patch Generation

Another important choice in the design of an AMR algorithm is the refinement criterion. This choice must be independent of the grid resolution so that the same refinement criterion, i.e., the same ε , will apply if the resolution is changed. An absolute difference criterion.

$$t_{i,j} = \max(\rho_{i+1,j} - \rho_{i-1,j}, \rho_{i,j+1} - \rho_{i,j-1}) > \varepsilon, \quad (33)$$

Directly measures the solution jump without being affected by Δx , making it more desirable than the normalised gradient criterion. Density is chosen as the indicator variable as any wave will have a corresponding density jump if it causes a jump in pressure or velocity.

The tagged cells are grouped into clusters of rectangular patches, which are controlled by two parameters: the grid efficiency λ and the blocking factor f , where the blocking factor ensures a minimum patch size to avoid excessive communication costs[7].

Time-Stepping and Conservation

Each level has its own CFL limited time step, so that the fine level does r sub-steps for each step on the coarse level, $\Delta t_\ell = \Delta t_0/r^\ell$. This sub-cycling, as noted by Berger and Colella[7], leads to a flux inconsistency between the coarse and fine grid cells, and this is corrected by refluxing, where the fine level fluxes, accumulated over all sub-steps, replace the corresponding coarse level fluxes on the faces, thus conserving mass, momentum, and energy globally. The coarse level cells in the patches on the fine grid are then replaced by the conservative average of the fine level data, thus ensuring consistency between the two levels before the next regridding step.

2.4. Adaptive Mesh Refinement via AMReX

The solver is provided as part of the AMReX library[2], which offers block-structured adaptive mesh refinement (AMR) on Cartesian grids, supporting MPI, OpenMP, and GPU parallelization. AMReX abstracts away the distinction between mesh management and method implementation: the user supplies the per-level advance routine and error tagging criteria, while inter-level communication, load balancing, and flux register bookkeeping are performed automatically by the library.

In this work, the AMR solver for the two-dimensional Euler equations is constructed upon the `AmrLevelAdv` class, which inherits from the `AmrLevel` class of the AMReX library. Three additional header files have been added for the physical problems: `EulerEquations.H` for the conserved quantities and the equation of state, `Reconstruction.H` for the MUSCL-Hancock method with limiter functions, and `RiemannSolver.H` for the HLLC approximate Riemann solver.

The conserved state $\mathbf{U} = (\rho, \rho u, \rho v, E)^\top$ is stored in `MultiFabs` with three ghost cells and filled before advancing with a call to `FillPatcherFill()` via interpolation from coarser levels and physical boundary conditions. In order to preserve conservation at the refinement boundaries, a `FluxRegister` is used to accumulate the difference between the coarse and fine grid interface fluxes, and after all the fine grid sub-cycles have been computed, the correction to the coarse grid is made by calling `reflux()`, and the cells on the coarse grid that correspond to the fine grid cells are overwritten by

the conservative average of the fine grid cells by calling `avgDown()`.

3. Results Part 1: One Dimensional Tests

The first test in this assignment corresponds to the tests in Toro's Table 4.1[1]. All tests use $\gamma = 1.4$ with initial data:

Table 1: Initial conditions for the one-dimensional Riemann test cases

Test	P_L	u_L	P_R	u_R	P_R	u_R	x_0	t_{end}
1 Sod	1.0	0.0	1.0	0.125	0.0	0.1	0.5	0.25
2 123	1.0	-2.0	0.4	1.0	2.0	0.4	0.5	0.15
3 Left Blast	1.0	0.0	1000.0	1.0	0.0	0.01	0.5	0.012
4 Right Blast	1.0	0.0	0.01	1.0	0.0	100.0	0.5	0.035
5 Collision	5.99924	19.5975	460.894	5.99242	-6.19633	46.0950	0.4	0.035

First, the solution will be tested at three different resolutions: 100×100 , 200×200 , and 400×400 . Next, the tests will be carried out at the lowest resolution, but the unrefined mesh will be used. In the last step, adaptive mesh refinement will be used to refine the mesh. It is important to ensure that smooth parts of the solution will not be refined. The convergence will also be tested for the unrefined mesh.

Due to the large number of tests, only the results for Test1 sod test and Test3 Left Blast will be shown. The results for the other three tests will be given in the appendix.

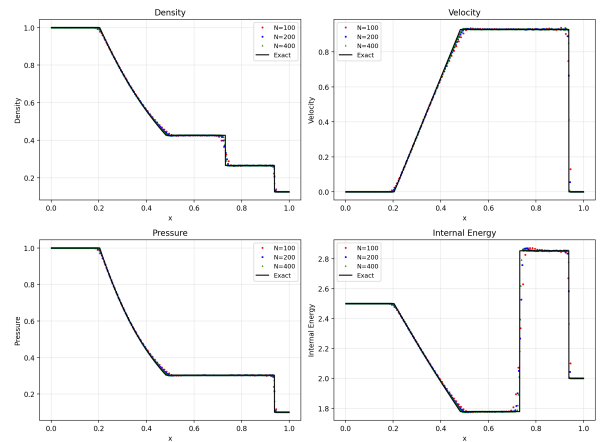


Figure 1: Toro Test 1: Sod Shock Tube ($t = 0.25$)

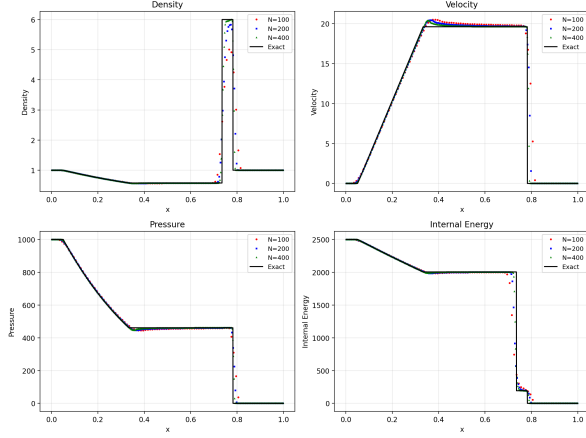


Figure 2: Toro Test 3: Left Blast Wave ($t = 0.012$)

In Figure 1, the Sod shock tube problem (Test 1) is shown at $t = 0.25$ with the three resolutions ($N = 100, 200, 400$) without the use of any Adaptive Mesh Refinement technique. The scheme is able to correctly resolve the entire structure of the three waves present in the problem, i.e., the left rarefaction fan, the contact discontinuity, and the right shock wave. The results can be seen to be converging to the exact solution as the resolution is increased.

In Figure 2, the left blast wave problem (Test 3) is shown at $t = 0.012$. This problem includes a much larger pressure ratio between the two states ($p_L/p_R = 10^5$). However, the results can be seen to be converging to the exact solution as the resolution is increased, with the results from the highest resolution ($N = 400$) almost coinciding with the exact solution.

Table 2: Error metrics for different test cases

Variable	Test 1 Sod	Test 2 123	Test 3 Left Blast	Test 4 Right Blast	Test 5 Collision
Density	0.90	0.84	0.80	0.81	0.97
Velocity	0.86	0.83	0.98	0.89	1.13
Pressure	0.99	0.92	0.97	0.89	0.97
Internal Energy	0.83	0.85	0.89	0.84	0.91

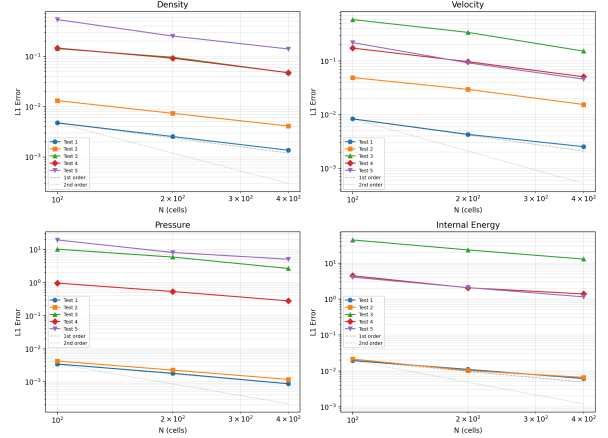


Figure 3: L_1 Error Convergence for Toro's Tests 1–5

Table 2 shows the convergence orders of all five test problems, from $N = 100$ to $N = 400$. All five test problems show convergence orders between 0.8 and 1.1. This is as expected, as it is a characteristic of first-order convergence of the schemes in problems with discontinuities. Although the MUSCL-Hancock method is second-order accurate, the presence of shocks, contact discontinuities, and edges of rarefaction waves limits the global L_1 convergence order of the method to be around first order.

Figure 3 shows plots of the L_1 errors of all five test problems as a function of resolution on a log-log plot. All five test problems' plots are nearly parallel to the reference plot of first-order convergence, showing the convergence orders reported in Table 1. Test 1 (Sod) has the lowest errors, while Tests 3 and 5 have the highest errors, as they have higher pressure ratios.

Now we considering apply AMR:

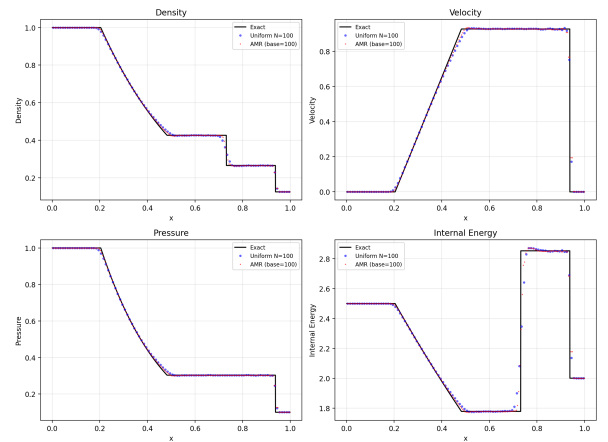


Figure 4: Toro's Test 1: AMR vs No-AMR $N=100$ ($t = 0.25$)

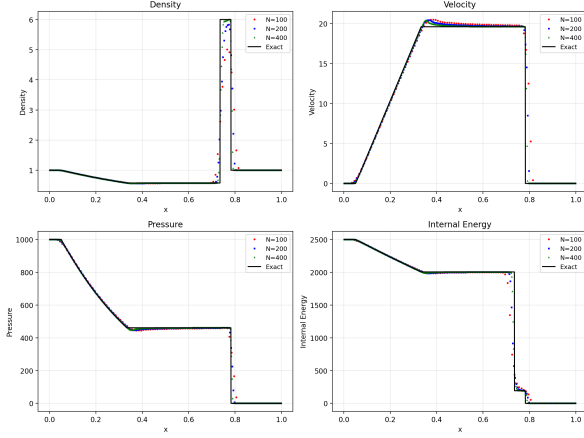


Figure 5: Toro's Test 3: AMR vs No-AMR $N=100$ ($t = 0.012$)

Figure 4 displays Test 1, Sod shock tube, with a comparison between AMR (with base $N = 100$, one level of refinement) and the solution with a uniform grid of $N = 100$. AMR has a more precise solution for the shock and contact discontinuity than the uniform grid solution, and the L_1 errors for this calculation are listed in Table 3.

Figure 5 shows Test 3, Left blast wave, with the same comparison between AMR and the solution on a uniform grid. Again, the extreme pressure ratio is handled well by AMR, which places more points at the shock front and rarefaction tail while leaving smooth regions at a coarse resolution. L_1 errors for this calculation are listed in Table 4.

Table 3: Test 1: Sod Shock Tube

Variable	Uniform $N = 100$	AMR (base= 100)	Reduction
Density	4.78e-3	3.13e-3	35%
Velocity	8.27e-3	6.60e-3	20%
Pressure	3.51e-3	2.50e-3	29%
Internal Energy	1.94e-2	1.38e-2	29%

Table 4: Test 3: Left Blast Wave

Variable	Uniform $N = 100$	AMR (base= 100)	Reduction
Density	1.42e-1	9.54e-2	33%
Velocity	5.64e-1	3.78e-1	33%
Pressure	9.80e+0	6.88e+0	30%
Internal Energy	4.43e+1	2.45e+1	45%

Tables 3 and 4 show the L_1 error comparison between the uniform solution with $N = 100$ and the AMR solution with base $N = 100$ and one refinement level for Test 1 and Test 3, respectively. For Test 1, we see an improvement in the L_1 error of 20–35% over all variables with the AMR solution. For Test 3, we see an

improvement of 30–45% with the largest being in the internal energy variable. Again, we see the solution obtained with the AMR with base $N = 100$ is as accurate as the solution obtained with $N = 200$ for the uniform solution.

Now we consider the AMR levels:

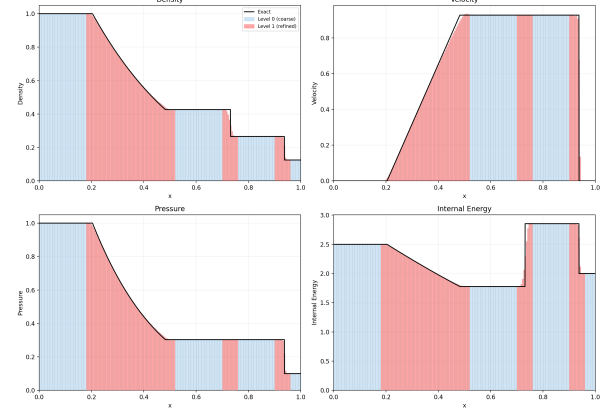


Figure 6: Toro's Test 1: AMR Grid Structure

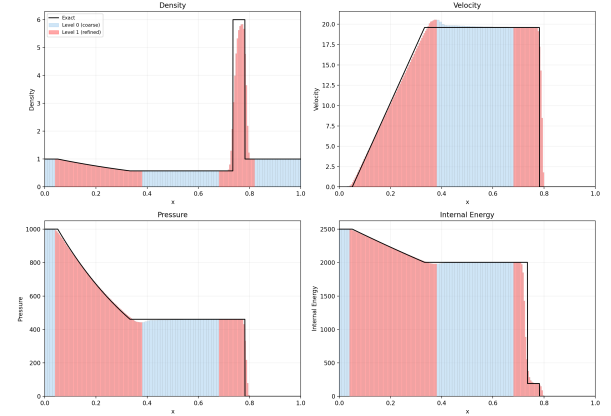


Figure 7: Toro's Test 3: AMR Grid Structure

Figures 6 and 7 demonstrate the structure of the AMR grid for Test 1 and Test 3, respectively, where each bar in the graph represents a computational cell, coloured according to its refinement level. Although the maximum refinement level is specified as 2, it is evident from the results of all five tests that only Level 1 refinement is triggered, as the density gradient at this level is not sufficient to trigger refinement at higher levels. For both Test 1 and Test 3, refinement is triggered at the discontinuity (shock, contact, rarefaction edge), while smooth regions are left at Level 0 refinement.

4. Results Part 2: Initial Two Dimensional Tests

In the present section, the one-dimensional Toro's test cases described in [1] are generalized to the two-dimensional case on the square domain $[0, 1] \times [0, 1]$. The simulations will be run with a uniform base resolution of 400×400 cells, which is sufficient to resolve all the structures of the waves present in the different test cases considered. Three different orientations of the discontinuity will be considered.

In the first two cases, the discontinuity will be aligned along the lines $x = 0.5$ and $y = 0.5$ respectively. These two configurations do not involve any variation along the perpendicular direction and should therefore reduce to the exact one-dimensional results, which can be considered a validation of the two-dimensional results. In the last case, the discontinuity will be diagonal, which will involve the fluxes along the x -direction as well as the y -direction simultaneously, thereby validating the two-dimensional results.

For all the cases considered here, the simulations will be run with or without the use of the adaptive mesh refinement method so as to be able to compare the results with the two different approaches at the same resolution.

4.1. Discontinuity along x

For Toro's test 1:

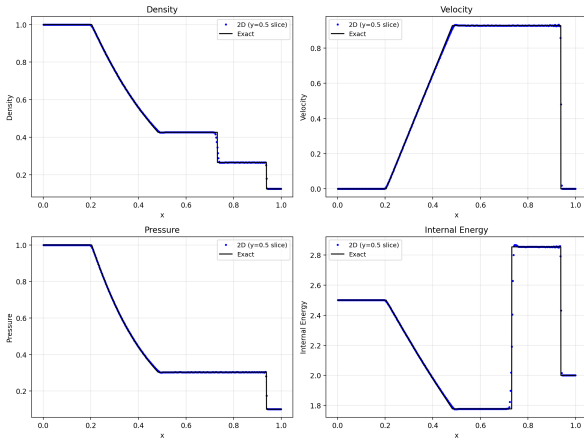


Figure 8: Toro's test 1 with discontinuity along $x = 0.5$

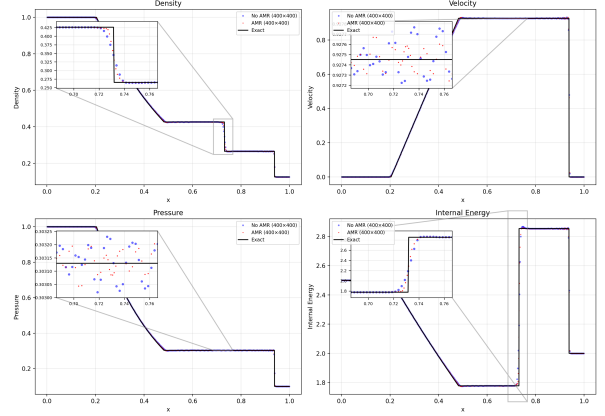


Figure 9: Toro's test 1: AMR and no-AMR comparison (x direction)

In Toro's test 1, the initial condition is the classical Sod shock tube problem, where the left state is $(\rho, u, p) = (1.0, 0.0, 1.0)$ and the right state is $(\rho, u, p) = (0.125, 0.0, 0.1)$. The solution is evolved until $t = 0.2$.

Figure 8 shows the density, velocity, pressure, and specific internal energy distributions using a 400×400 grid. The results match the one-dimensional wave pattern of a left rarefaction, contact discontinuity, and a right-moving shock, but without any variation in the y -direction, consistent with the one-dimensional results.

Figure 9 shows the results of the corresponding AMR computation, where refinement is concentrated around the shock and contact discontinuity, while smooth regions are at the base resolution of the grid. The results are very close to the corresponding results of the uniform grid computation, but with a better definition of the contact discontinuity and shock front because of the increased resolution in these regions.

For Toro's test 3:

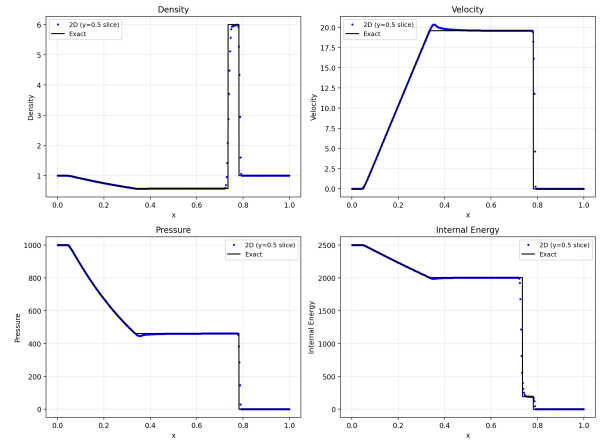


Figure 10: Toro's test 3 with discontinuity along $x = 0.5$

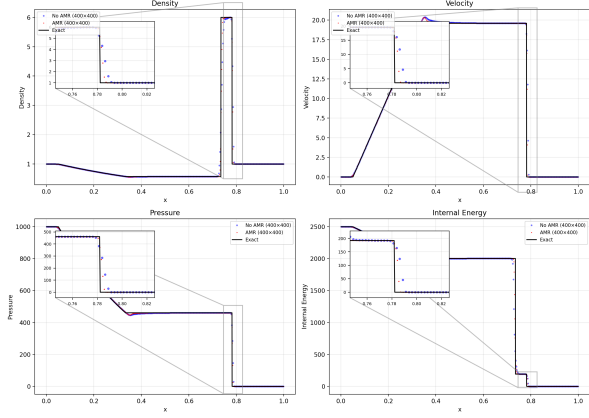


Figure 11: Toro's test 3: AMR and no-AMR comparison (x direction)

For Toro's test 3, Figures 10 and 11 illustrate the results with discontinuity along $x = 0.5$ at $t = 0.012$. The initial condition results in a stronger wave structure than test 1, with a left rarefaction, a contact discontinuity, and a right-travelling shock of greater strength. As with test 1, the results do not exhibit any variation in the y -direction, thus demonstrating the results to be consistent with the 1D results. The results from the Adaptive Mesh Refinement procedure shown in Figure 11 exhibit a slightly sharper shock resolution than the uniform grid results.

4.2. Discontinuity along y

The tests are then repeated with the initial discontinuity placed at $y = 0.5$, so that the left state is $y < 0.5$ and the right state is $y > 0.5$. By symmetry, we expect the results to be the same as those found in Section 4.1, and any difference would suggest an asymmetry in the x and y flux routines.

For Toro's test 1:

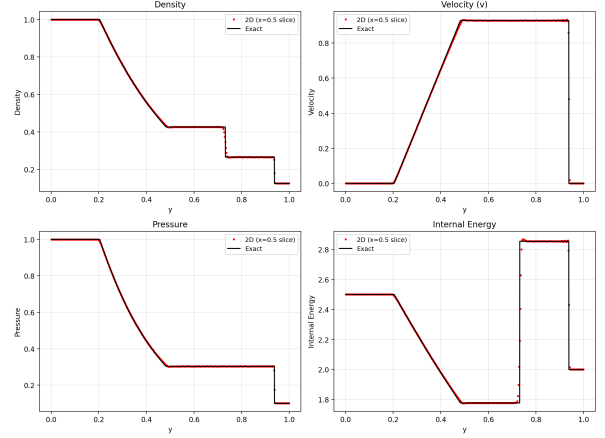


Figure 12: Toro's test 1 with discontinuity along $y = 0.5$

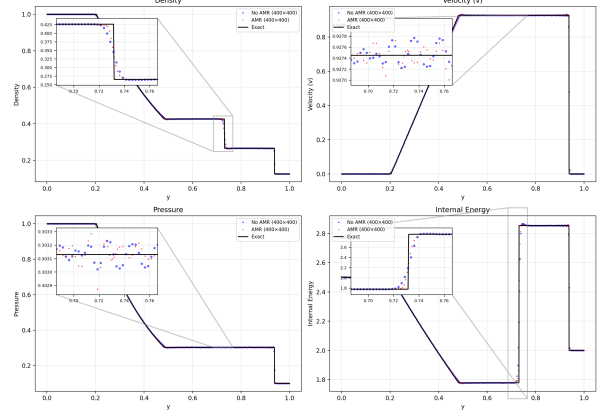


Figure 13: Toro's test 1: AMR and no-AMR comparison (y direction)

Figure 12 demonstrates the uniform grid solution for Toro's test 1 and compares well with the x -direction solution in Figure 8. This further confirms the isotropic nature of our solver. Figure 13 demonstrates the AMR solution to this problem and compares well with the uniform grid solution with a slightly sharper representation of the contact and shock front.

For Toro's test 3:

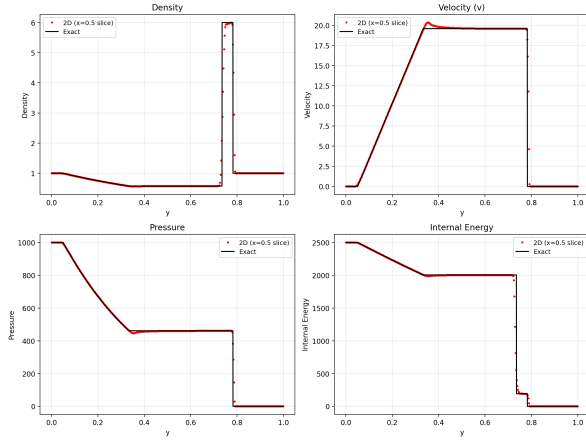


Figure 14: Toro's test 3 with discontinuity along $y = 0.5$

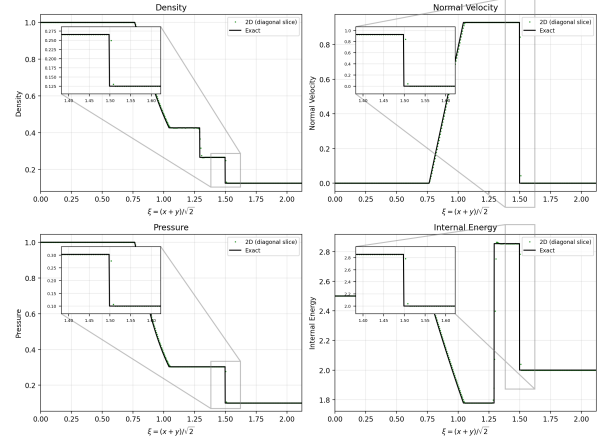


Figure 16: Toro's test 1 with discontinuity along diagonal

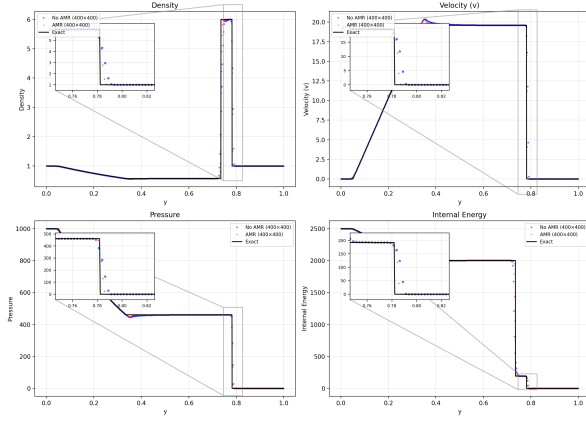


Figure 15: Toro's test 3: AMR and no-AMR comparison (y direction)

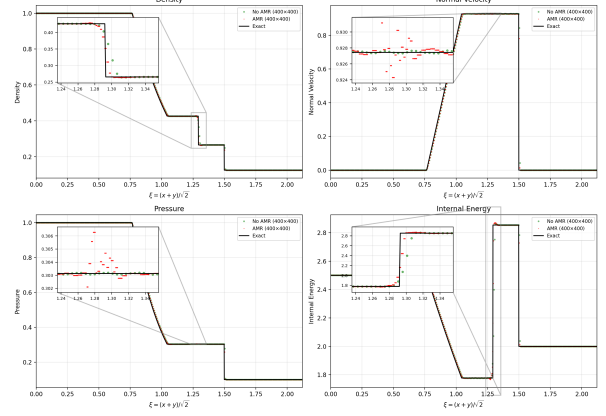


Figure 17: Toro's test 1: AMR and no-AMR comparison (diagonal direction)

This same phenomenon is seen in Toro's test 3 in Figures 14 and 15, where the much stronger wave structures caused by the extreme pressure ratio used ($p_L/p_R = 10^5$) are correctly predicted in the uniform grid and AMR simulations. The solution does not vary in the x direction, confirming the symmetry of the code between the two coordinate directions. Even with the more difficult initial conditions, the AMR solution is stable and correctly resolves the refinement near the shock front and rarefaction tail, similar to the results seen in Section 4.1.

4.3. Diagonal discontinuity

For Toro's test 1:

For Toro's test 3:

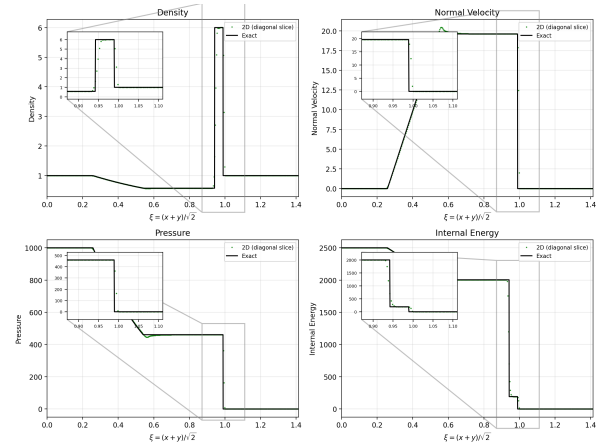


Figure 18: Toro's test 3 with discontinuity along diagonal

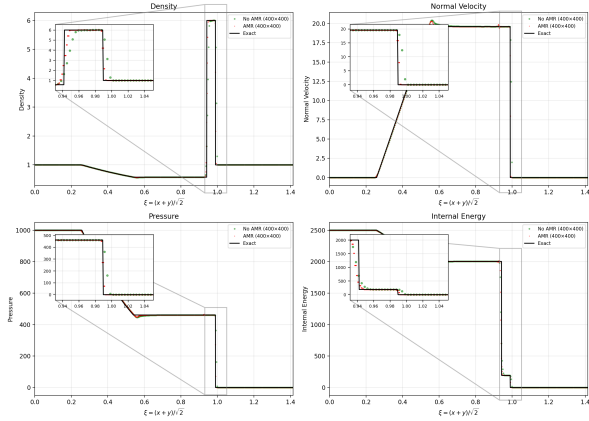


Figure 19: Toro's test 3: AMR and no-AMR comparison (diagonal direction)

In Toro's test 1, Figure 16 displays the density, velocity, pressure, and specific internal energy on a uniform 400×400 grid with the discontinuity oriented diagonally and plotted against the normal coordinate $\xi = \frac{x+y}{\sqrt{2}}$. The solution correctly captures the one-dimensional wave structure consisting of a left rarefaction, a contact discontinuity, and a right-travelling shock, as seen in the results in Sections 4.1 and 4.2. This verifies that there is no directional bias in the implementation of the two-dimensional flux. Figure 17 shows the AMR solution for this problem, with the solution being correctly concentrated on the oblique discontinuity and being in close agreement with the uniform grid solution.

The same phenomenon is also observed in Toro's test 3, as seen in Figures 18 and 19, where the strong wave structures are correctly captured in the uniform and AMR solutions.

5. Results Part 3: Two Dimensional Timing Exercise

In this section, the two-dimensional Riemann problem, as proposed by Lax and Liu[8], is employed to assess the performance characteristics of the code. The initial conditions consist of four distinct states with discontinuities along $x = 0.5$ and $y = 0.5$, which are advanced to a final time of $t = 0.3$. The objective is to measure the improvement in simulation time due to resolution, parallelism, and adaptivity, as well as to verify the improvement in accuracy with resolution and related parameters.

The initial conditions are as follows, with a final time of $t = 0.3$ [8]:

$$\mathbf{w} = (\rho, v_x, v_y, p)^T = \begin{cases} (1.5, 0, 0, 1.5)^T & x > 0.5, y > 0.5 \\ (0.5323, 1.206, 0, 0.3)^T & x < 0.5, y > 0.5 \\ (0.138, 1.206, 1.206, 0.029)^T & x < 0.5, y < 0.5 \\ (0.5323, 0, 1.206, 0.3)^T & x > 0.5, y < 0.5 \end{cases} \quad (34)$$

5.1. Resolution study

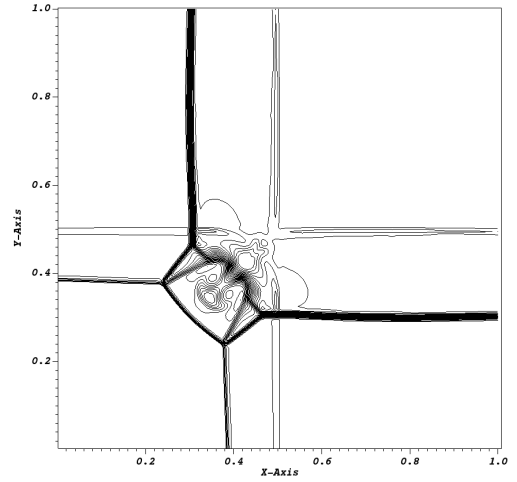


Figure 20: Density contour at $t = 0.3$ with 200×200 Grid

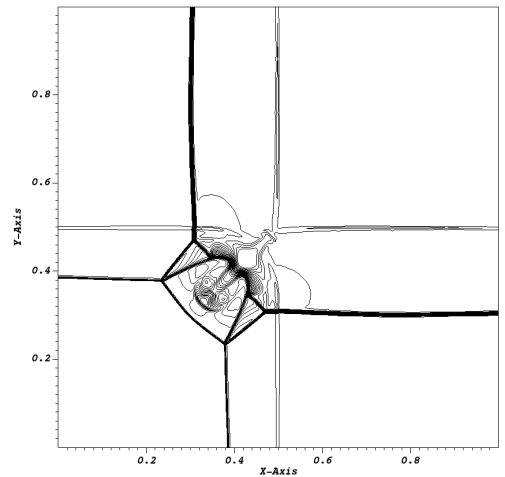


Figure 21: Density contour at $t = 0.3$ with 400×400 Grid

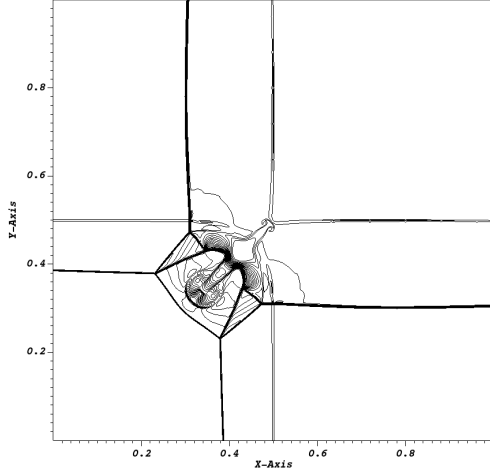


Figure 22: Density contour at $t = 0.3$ with 800×800 Grid

Table 5: Wall-clock time at different resolutions

Resolution	Wall time (s)
200×200	167.9
400×400	1368.5
800×800	11063.5

Table 5 indicates the simulation times for the three resolutions on a single CPU. It is evident that, as the resolution doubles from 200 to 400 to 800, the simulation time increases by a factor of nearly 8 for each case, which is consistent with the expected scaling for a two-dimensional simulation, where doubling the resolution doubles the number of cells in each direction and halves the time step. Figures 20, 21, and 22 which are drawn by Visit show the density contour plots at $t=0.3$ for the three different resolutions.

5.2. MPI parallelisation

The same simulations as in Section 5.1 are performed using 1, 2, 4, 8, and 16 CPU cores to measure the effect of using MPI parallelization on the wall time.

Table 6: Simulation wall times (s) for different resolutions and CPU core counts

Resolution	1 core	2 cores	4 cores	8 cores	16 cores
200×200	167.9	89.4	50.5	30.5	14.9
400×400	1368.5	706.1	363.5	185.4	115.5
800×800	11063.5	5591.3	2818.3	1464.8	790.9

Table 7: Parallel speedup for different resolutions and CPU core counts

Resolution	1 core	2 cores	4 cores	8 cores	16 cores
200×200	1.00	1.88	3.32	5.50	11.3
400×400	1.00	1.94	3.76	7.38	11.8
800×800	1.00	1.98	3.92	7.55	14.0

Table 6 shows the simulation times for the three resolutions with 1, 2, 4, 8, and 16 CPU cores. Table 7 summarises the speedup factors for all resolutions and core counts. Increasing the number of cores consistently reduces the simulation time for each of the three resolutions. For the 200×200 grid, the simulation time is reduced from 167.9 seconds on 1 core to 14.9 seconds on 16 cores, achieving a speedup of 11.3. For the 400×400 grid, the simulation time is reduced from 1368.5 seconds on 1 core to 115.5 seconds on 16 cores, achieving a speedup of 11.8. For the 800×800 grid, the simulation time is reduced from 11063.5 seconds on 1 core to 790.9 seconds on 16 cores, achieving a speedup of 14.0. Increasing the resolution improves the speedup, as the increased workload per core reduces the overhead of inter-process communication. For all three grids, the speedup is less than the ideal linear speedup of 16, which is expected as there is overhead in MPI parallelisation at domain boundaries and some parts of the problem cannot be parallelised. However, the results show that MPI parallelisation is effective in reducing the wall time, especially for the highest resolution where the computational cost is largest.

5.3. AMR Efficiency and Speedup

The lowest resolution of 200×200 will be taken as a starting grid and AMR will be applied to attain effective resolutions comparable to higher uniform grid cases presented in Section 5.1, and also to study the effect of MPI parallelisation with AMR to attain maximum speedup.

Table 8: Comparison of simulation times (seconds) for uniform-grid and AMR runs on a single CPU.

Configuration	Effective resolution	Time (s)
200×200 , no AMR	200×200	167.9
200×200 + 1 level	400×400	79.0
200×200 + 2 levels	800×800	80.8
400×400 , no AMR	400×400	1368.5
800×800 , no AMR	800×800	11063.5

A comparison of the simulation times for the uniform grid and the AMR grid on a single CPU is shown in Table 8. The use of a base grid size of 200×200 with a

single refinement level results in a grid size of 400×400 with a computation time of 79.0s compared to 1368.5 s for the uniform grid size of 400×400 . This is a speedup of approximately 17 times. The addition of a second refinement level to give a grid size of 800×800 results in a computation time of 80.8s compared to 11063.5s for the uniform grid size of 800×800 . This is a speedup of approximately 137 times. This shows the effectiveness of the AMR method as the wave structures are small compared to the domain size, so the majority of the domain can be computed with a coarse grid without compromising the results. The use of the AMR method with the MPI parallelisation reduces the computation times even further. For example, the use of a base size of 200×200 with two refinement levels results in a grid size of 800×800 with a computation time of 8.4s compared to 790.9s for the uniform grid size of 800×800 on 16 CPUs. This represents the fastest speedup achieved in this work, a factor of approximately 94 over the single core uniform 800×800 simulation (11063.5 s).

5.4. AMR parameter study

The impact of the three AMR parameters `amr.blocking_factor`, `amr.n_error_buf`, and `tagging.phigrad` on simulation time is studied using the 200×200 base grid with two levels of refinements.

Table 9: Effect of AMR parameters on simulation time.

Parameter	Value	Time (s)
Baseline	—	80.8
<code>amr.blocking_factor</code>	8	84.1
<code>amr.n_error_buf</code>	1	80.1
<code>amr.n_error_buf</code>	2	86.2
<code>amr.n_error_buf</code>	4	93.8
<code>tagging.phigrad</code>	0.3	88.4
<code>tagging.phigrad</code>	0.8	51.0

In Table 9, we demonstrate the effect of changing three different AMR parameters on the simulation time. The baseline case is the 200×200 grid, two levels of refinement, and the baseline time is 80.8s. The parameters we have chosen to vary are `amr.blocking_factor`: the minimum patch size for the AMR, `amr.n_error_buf`: the number of cells to add to the buffer region, and `tagging.phigrad`: the gradient value for refinement.

Increasing the value of the `amr.blocking_factor` to 8 increases the simulation time to 84.1s. This is because the shape of the refined region is constrained to

be a multiple of the `blocking_factor`, and so slightly more cells may be refined than needed.

By adjusting `amr.n_error_buf` to be 4 instead of 1, the simulation time increases from 80.1s to 93.8s. This is because a larger error buffer means that more cells surrounding the flagged region are refined, thus increasing the total number of fine cells. This is a trade-off between the robustness of the solution, such that the wave structures do not propagate outside the refined region between regridding times, and the efficiency of the computation.

The gradient refinement threshold `tagging.phigrad` is the most significant factor when adjusting the parameters to see how the simulation time is affected. By adjusting the gradient refinement threshold to 0.3, the simulation time is increased to 88.4s as more cells across the domain satisfy the refinement criteria and thus become refined. However, increasing the gradient refinement threshold to 0.8 decreases the simulation time to 51.0s as fewer cells become refined.

6. Conclusions

In this report, a second-order finite volume scheme for the compressible Euler equations in one dimension has been implemented using the AMReX framework with a MUSCL-Hancock method and some Riemann solvers. First, the code was validated against one-dimensional Toro test cases and shown to converge with increasing resolution. The code was then extended to two dimensions and tested against a set of two-dimensional tests with discontinuities oriented along the x and y axes to verify that the two-dimensional results matched the one-dimensional results. Additionally, a diagonal orientation test verified that the two-dimensional flux calculation was correct. For the x and y axis tests, a slightly sharper representation of the shock and contact discontinuity was achieved with the AMR solution compared to the uniform grid solution at an equivalent level of resolution, showing that AMR can provide a more refined representation of features while avoiding error at coarse/fine boundaries.

The computational efficiency of the solver was also examined through the two-dimensional Riemann problem of Lax and Liu. In the absence of AMR, the simulation time scales like 2^3 when the resolution is doubled. This is the expected scaling for a two-dimensional explicit scheme. MPI parallelisation resulted in considerable reductions in wall time. In particular, a maximum speedup of approximately 14 was obtained when using 16 cores for the simulation on the 800×800 grid.

AMR was also found to be extremely efficient. In particular, a simulation on a 200×200 grid with two levels of AMR resulted in a simulation time of 80.8s on a single CPU, whereas the equivalent simulation on a uniform grid resulted in a simulation time of 11063s. Thus, the simulation time was reduced by a factor of approximately 137. Furthermore, the combination of AMR with 16 MPI processes resulted in a simulation time of 8.4s. The AMR parameter study indicated that the parameter `tagging.phigrad` that controls the gradient threshold has the most significant impact on the simulation time.

Some possible areas for further research that come to mind are as follows. The current gradient-based tagging criterion can be replaced by a more stringent error-based tagging criterion, which would give more control over the accuracy of the solution. The code can be generalized to three dimensions, in which case the advantages of AMR over uniform grids would be more pronounced. Higher-order schemes, such as WENO, can be employed for more accurate resolution of smooth features while preserving shock-capturing ability. Also, if we use other second order numerical method maybe will get different conclusions.

Appendix A. Initial One dimensional Test

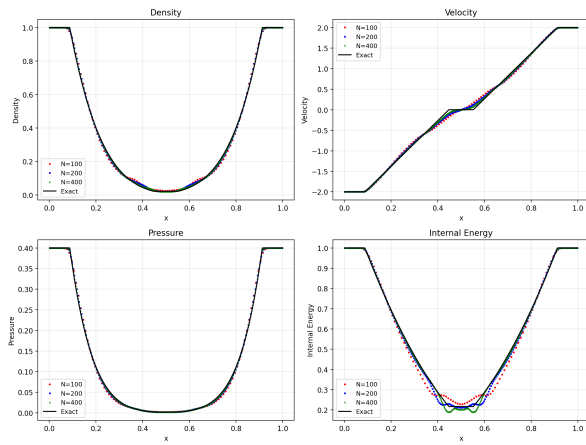


Figure A.23: Toro's Test 2: 123 Problem ($t = 0.15$)

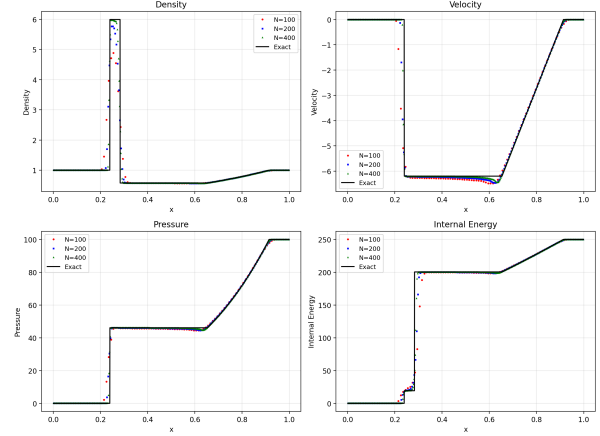


Figure A.24: Toro's Test 4: Right Blast Wave ($t = 0.035$)

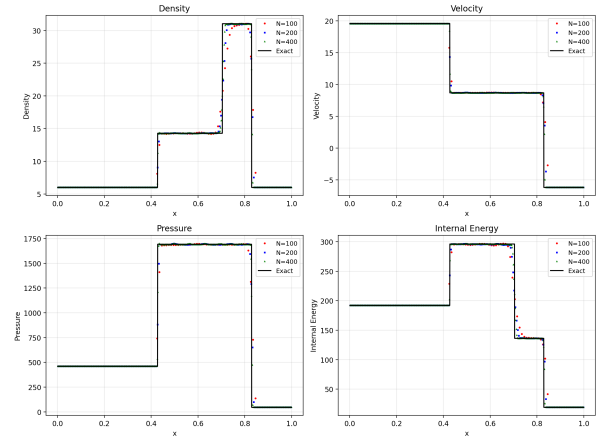


Figure A.25: Toro's Test 5: Two-Shock Collision ($t = 0.035$)

Table A.10: L1 errors for different variables and test cases

	Test 1 Sod	Test 2 123	Test 3 Left Blast	Test 4 Right Blast	Test 5 Collision
Density L1 (N=100)	4.78e-3	1.32e-2	1.42e-1	1.46e-1	5.38e-1
Density L1 (N=400)	1.36e-3	4.13e-3	4.72e-2	4.73e-2	1.40e-1
Velocity L1 (N=100)	8.36e-3	4.91e-2	5.93e-1	1.74e-1	2.21e-1
Velocity L1 (N=400)	2.53e-3	1.55e-2	1.53e-1	5.08e-2	4.60e-2
Pressure L1 (N=100)	3.46e-3	4.26e-3	1.03e+1	9.71e-1	1.96e+1
Pressure L1 (N=400)	8.76e-4	1.18e-3	2.68e+0	2.81e-1	5.12e+0
Int Energy L1 (N=100)	1.94e-2	2.14e-2	4.42e+1	4.45e+0	4.06e+0
Int Energy L1 (N=400)	6.13e-3	6.61e-3	1.29e+1	1.39e+0	1.15e+0

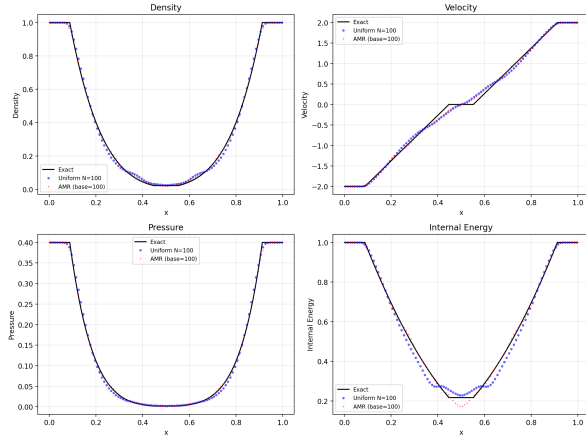


Figure A.26: Toro's Test 2: AMR vs No-AMR N=100 ($t = 0.15$)

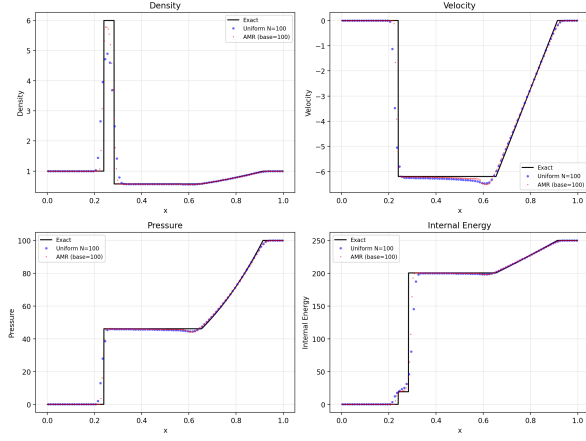


Figure A.27: Toro's Test 4: AMR vs No-AMR N=100 ($t = 0.035$)

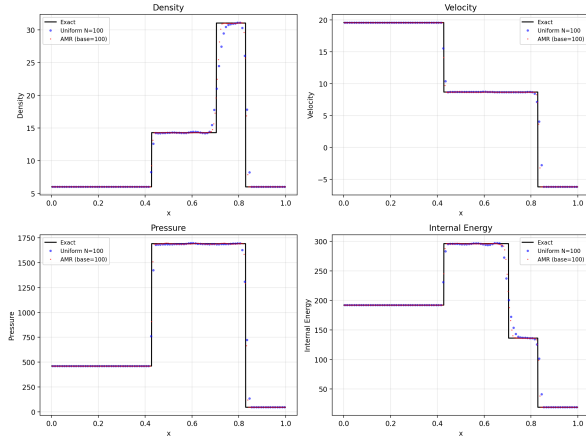


Figure A.28: Toro's Test 5: AMR vs No-AMR N=100 ($t = 0.035$)

Table A.11: L1 error comparison (Uniform $N = 100$ vs AMR base=100)

	Test 1 Sod	Test 2 123	Test 3 Left Blast	Test 4 Right Blast	Test 5 Collision
Density Uniform	4.78e-3	1.32e-2	1.42e-1	1.45e-1	5.29e-1
Density AMR	3.13e-3	7.91e-3	9.54e-2	9.17e-2	2.58e-1
Velocity Uniform	8.27e-3	4.91e-2	5.64e-1	1.66e-1	2.17e-1
Velocity AMR	6.60e-3	3.20e-2	3.78e-1	1.09e-1	9.64e-2
Pressure Uniform	3.51e-3	4.26e-3	9.80e+0	9.20e-1	1.94e+1
Pressure AMR	2.50e-3	2.74e-3	6.88e+0	6.41e-1	8.41e+0
Int Energy Uniform	1.94e-2	2.14e-2	4.43e+1	4.46e+0	4.03e+0
Int Energy AMR	1.38e-2	7.51e-3	2.45e+1	2.17e+0	2.19e+0

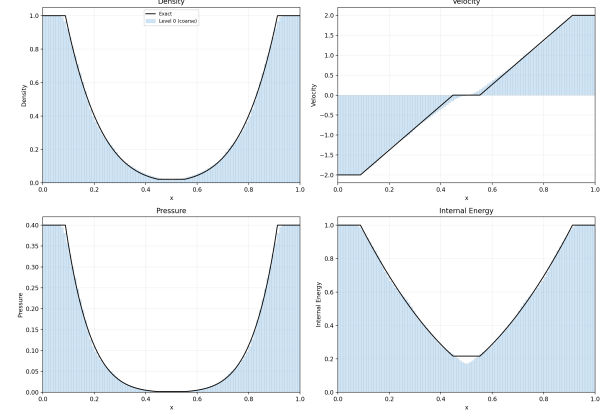


Figure A.29: Toro's Test 2: AMR Grid Structure

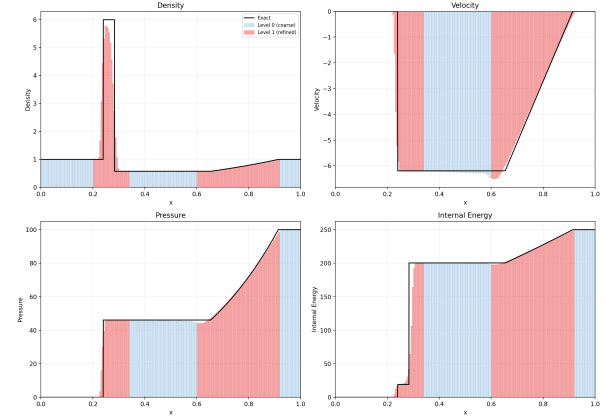


Figure A.30: Toro's Test 4: AMR Grid Structure

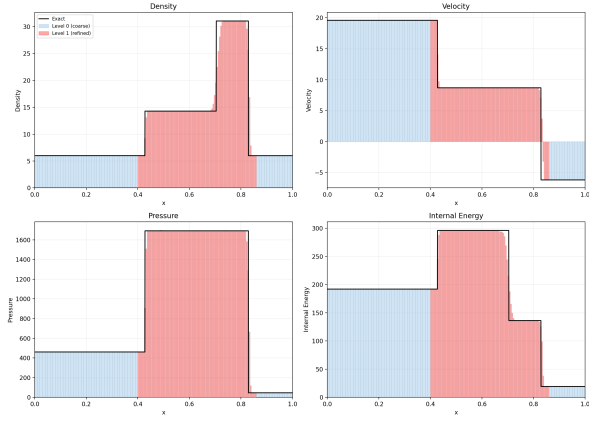


Figure A.31: Toro's Test 5: AMR Grid Structure

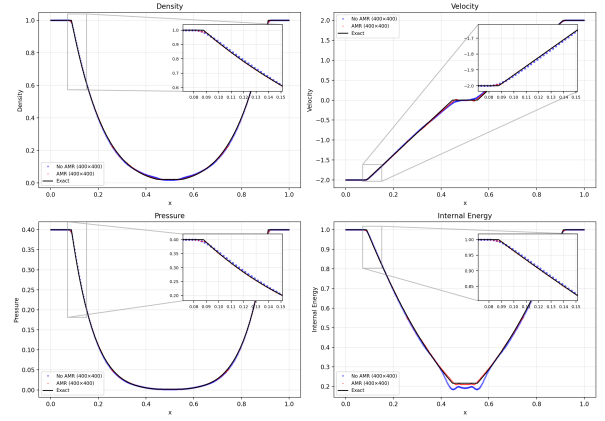


Figure B.34: Toro's test 2: AMR and no-AMR comparison (x direction)

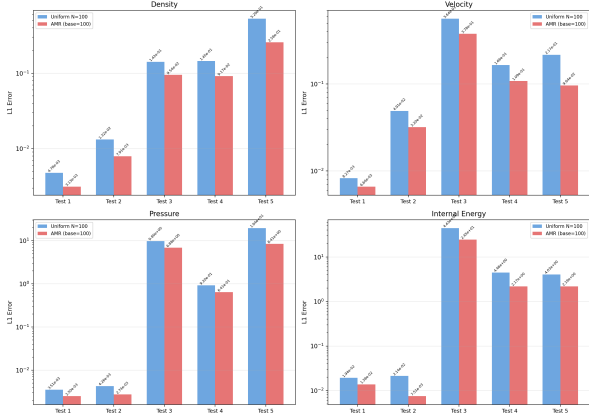


Figure A.32: One dimensional tests' L_1 error

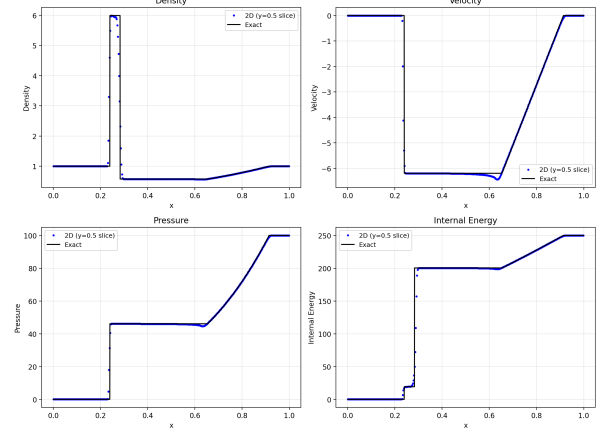


Figure B.35: Toro's test 4 with discontinuity along $x = 0.5$

Appendix B. Initial two dimensional tests

Appendix B.1. Discontinuity along x

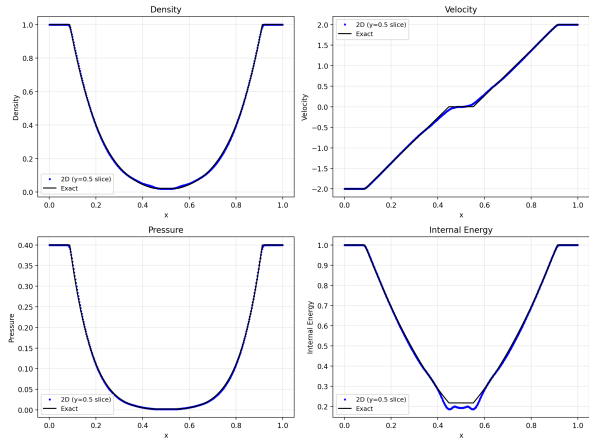


Figure B.33: Toro's test 2 with discontinuity along $x = 0.5$

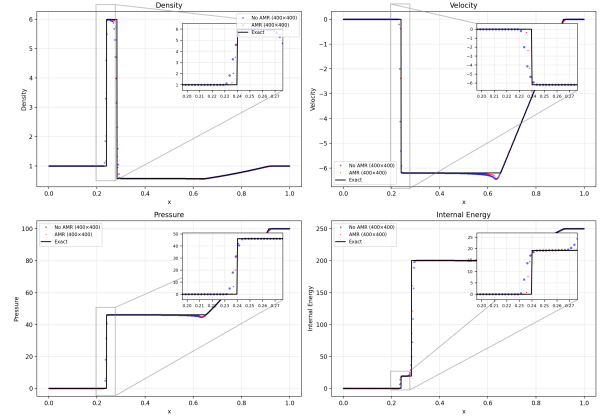


Figure B.36: Toro's test 4: AMR and no-AMR comparison (x direction)

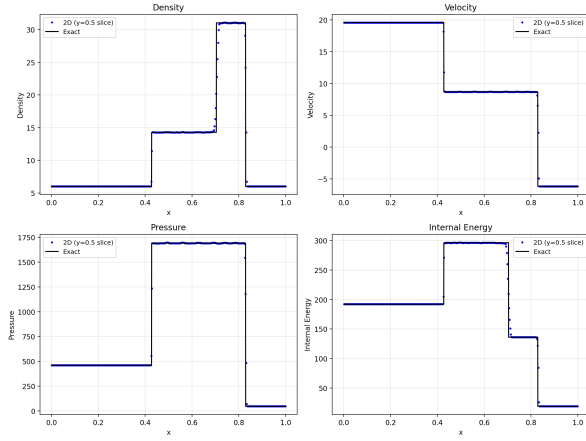


Figure B.37: Toro's test 5 with discontinuity along $x = 0.5$

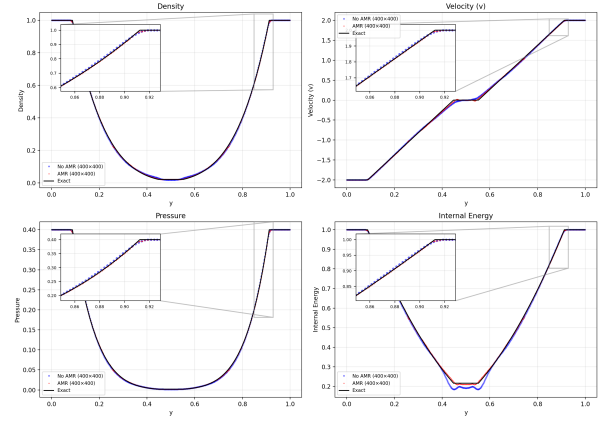


Figure B.40: Toro's test 2: AMR and no-AMR comparison (y direction)

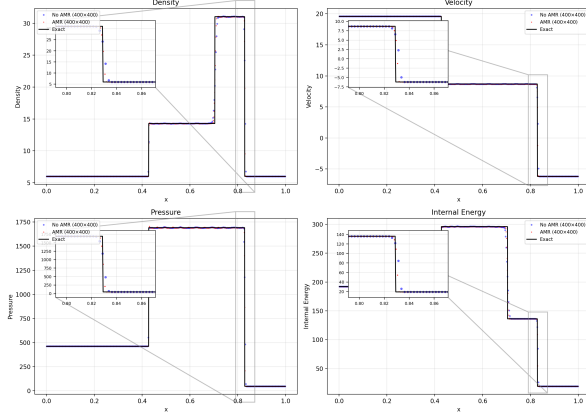


Figure B.38: Toro's test 5: AMR and no-AMR comparison (x direction)

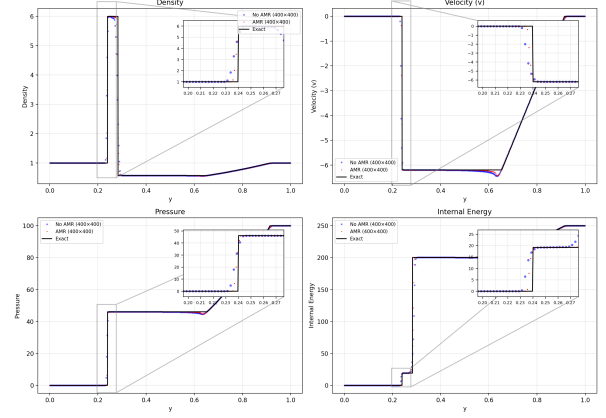


Figure B.41: Toro's test 4 with discontinuity along $y = 0.5$

Appendix B.2. Discontinuity along y

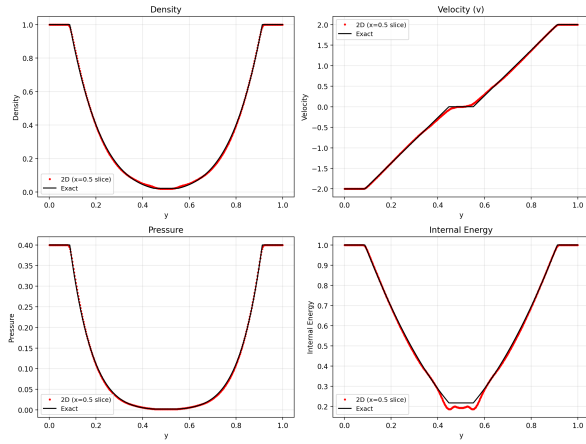


Figure B.39: Toro's test 2 with discontinuity along $y = 0.5$

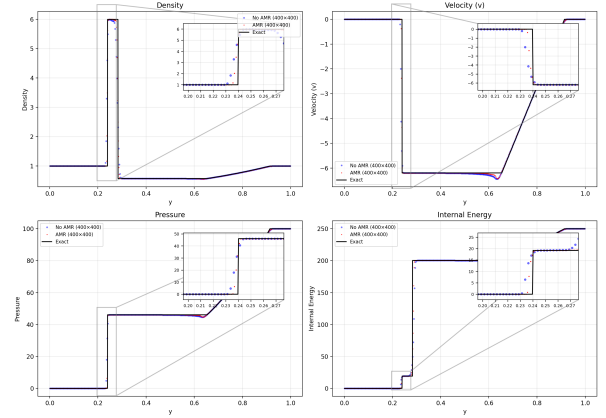


Figure B.42: Toro's test 4: AMR and no-AMR comparison (y direction)

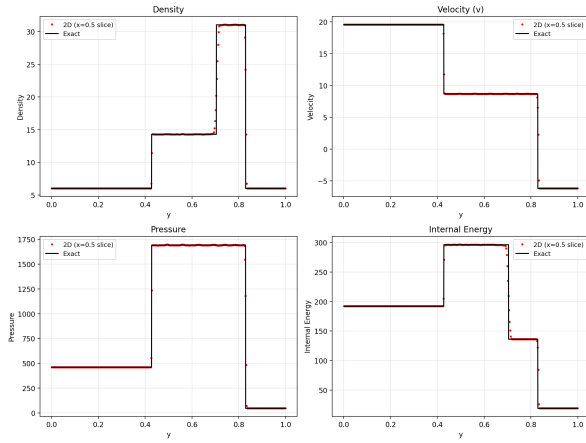


Figure B.43: Toro's test 5 with discontinuity along $y = 0.5$

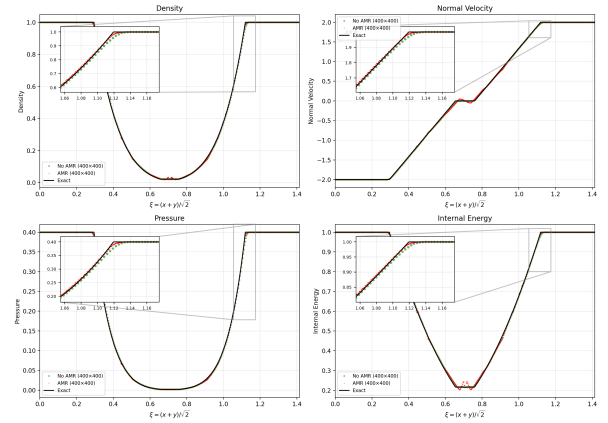


Figure B.46: Toro's test 2: AMR and no-AMR comparison (diagonal direction)

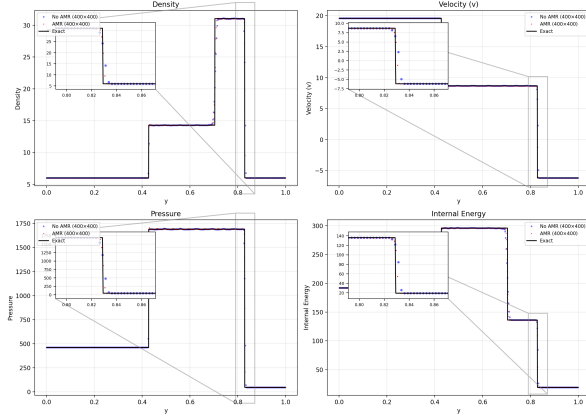


Figure B.44: Toro's test 5: AMR and no-AMR comparison (y direction)

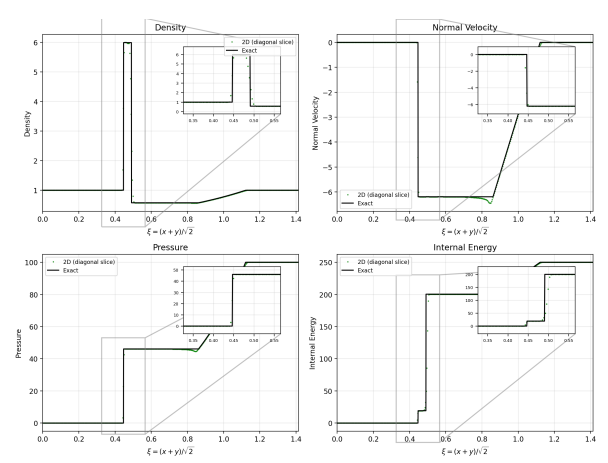


Figure B.47: Toro's test 4 with discontinuity along diagonal

Appendix B.3. Diagonal discontinuity

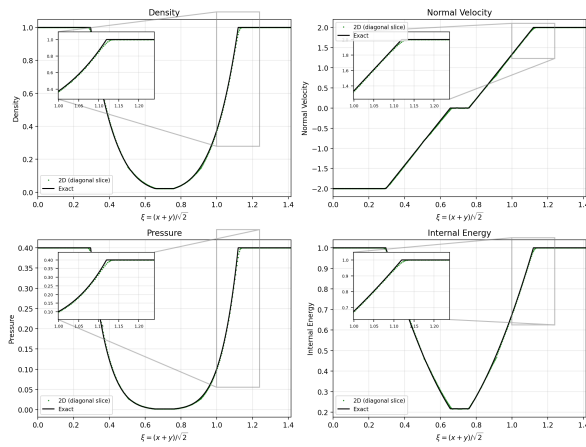


Figure B.45: Toro's test 2 with discontinuity along diagonal

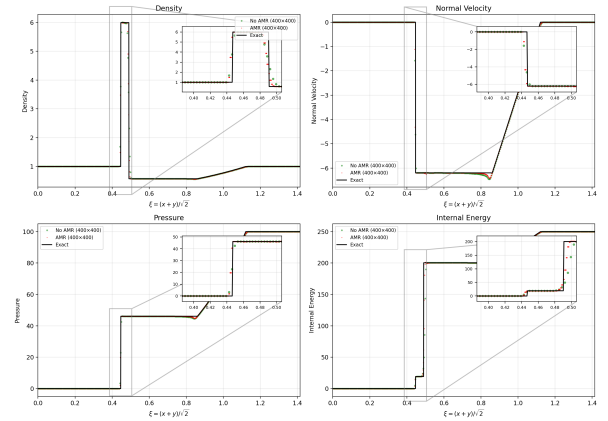


Figure B.48: Toro's test 4: AMR and no-AMR comparison (diagonal direction)

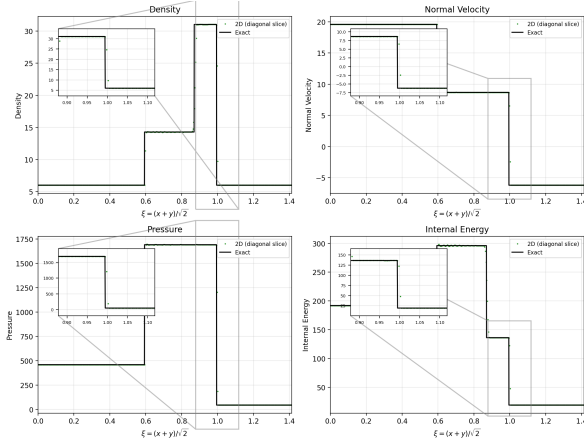


Figure B.49: Toro's test 5 with discontinuity along diagonal

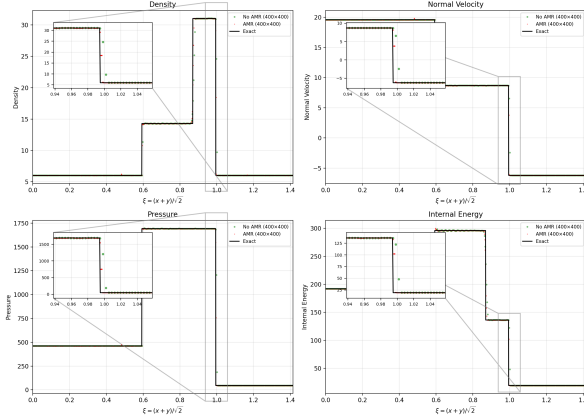


Figure B.50: Toro's test 5: AMR and no-AMR comparison (diagonal direction)

Appendix B.4. Mesh Patches

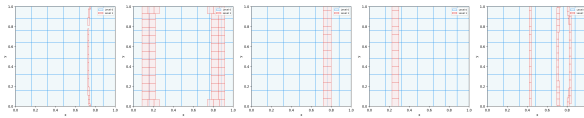


Figure B.51: AMR Mesh Patches — X Direction

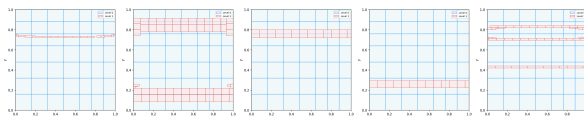


Figure B.52: AMR Mesh Patches — Y Direction

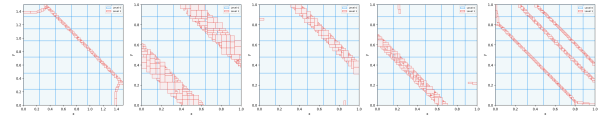


Figure B.53: AMR Mesh Patches — Diagonal Direction

References

- [1] E. F. Toro, Riemann solvers and numerical methods for fluid dynamics: a practical introduction, Springer Science & Business Media, 2013.
- [2] W. Zhang, A. Almgren, V. Beckner, J. Bell, J. Blaschke, C. Chan, M. Day, B. Friesen, K. Gott, D. Graves, et al., Amrex: a framework for block-structured adaptive mesh refinement, The Journal of Open Source Software 4 (37) (2019) 1370.
- [3] M. J. Berger, P. Colella, Local adaptive mesh refinement for shock hydrodynamics, Journal of computational Physics 82 (1) (1989) 64–84.
- [4] B. Van Leer, Towards the ultimate conservative difference scheme iii. upstream-centered finite-difference schemes for ideal compressible flow, Journal of Computational Physics 23 (3) (1977) 263–275.
- [5] S. Tang, Adaptive symmetric flux limiters with long computation times for hyperbolic conservation laws, Results in Applied Mathematics 20 (2023) 100410.
- [6] P. Colella, Multidimensional upwind methods for hyperbolic conservation laws, Journal of computational physics 87 (1) (1990) 171–200.
- [7] M. J. Berger, J. Olinger, Adaptive mesh refinement for hyperbolic partial differential equations, Journal of computational Physics 53 (3) (1984) 484–512.
- [8] P. D. Lax, X.-D. Liu, Solution of two-dimensional riemann problems of gas dynamics by positive schemes, SIAM Journal on Scientific Computing 19 (2) (1998) 319–340.